ClearQuote SDK – Flutter Integration Guide

1. Introduction

This document describes how to integrate the ClearQuoteSDK into a Flutter application using platform channels. The SDK enables vehicle inspection flows, including camera capture, offline inspections, and inspection completion callbacks.

Supported platforms:

- iOS — Integrated (ClearQuoteSDK via Swift Package Manager + ClearQuotePlugin.swift)
- Android — Not Implemented yet

2. Prerequisites

Before starting the integration, ensure the following requirements are met:

Common

- Flutter SDK: 3.44+ / Dart 3.12+
- A valid ClearQuote SDK Key (contact the ClearQuote team)

2.1 iOS

- Supported Device: iPhone
- Minimum iOS Deployment Target: iOS 16.0
- Xcode Version: 16.0 or later
- Swift Package Manager support in the iOS project
- Swift support must be enabled in the iOS project

2.2 Android

Not Implemented yet

Android ClearQuoteSDK Flutter bridging is not available yet. Android-specific prerequisites will be documented when support is added.

3. Architecture

Flutter talks to ClearQuoteSDK through platform channels:

```
Flutter UI (Dart)
  ↕ MethodChannel  com.clearquote/sdk
  ↕ EventChannel   com.clearquote/sdk/events
  ├─ iOS     → ClearQuotePlugin.swift → ClearQuoteSDK (SPM)
  └─ Android → Not Implemented yet
```

4. Install ClearQuote SDK

4.1 iOS

Step 1: Add ClearQuoteSDK via Swift Package Manager

In Xcode, open ios/Runner.xcworkspace, then:

- File → Add Package Dependencies…
- Add Repository URL: https://github.com/clearquotetech/cq-ios-sdk.git

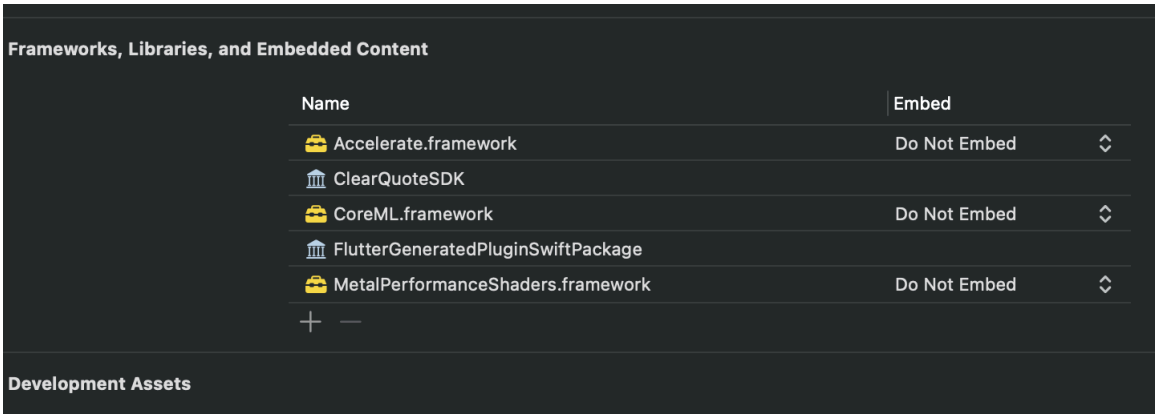
```swift
.package(
    url: "https://github.com/clearquotetech/cq-ios-sdk.git",
    .branch("main")
)// Xcode 27 & above

.package(
    url: "https://github.com/clearquotetech/cq-ios-sdk.git",
    .branch("xcode26")
)// Xcode 26 (Supporting till End of 2026)
```

- Add the ClearQuoteSDK product to your app target

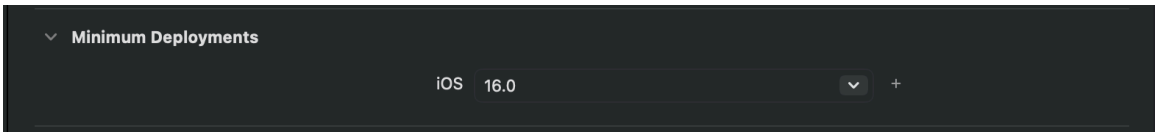
Also link these Apple frameworks on the Runner target for SDK runtime:

- CoreML
- MetalPerformanceShaders
- Accelerate



Step 2: Set iOS Deployment Target

Ensure IPHONEOS_DEPLOYMENT_TARGET is set to 16.0 for the Runner project and targets.



Step 3: Update Info.plist

- Camera permission

```xml
<key>NSCameraUsageDescription</key>
<string>For capturing vehicle images</string>
```

- Location permission

If using location feature from the SDK then based on **locationUsage** (**foregroundOnly** OR **withBackground**) selected, set the relevant info.plist keys

```swift
ClearQuote.shared.locationUsage = .foregroundOnly/.withBackground
// Set this in didFinishLaunchingWithOptions of AppDelegate if want to use location feature. By default, locationUsage is disabled

// foregroundOnly - location is used while using the app
// withBackground - location is used while using the app & also when the app is in background
```

- If **locationUsage** is **foregroundOnly**

Key: Privacy - Location When In Use Usage Description

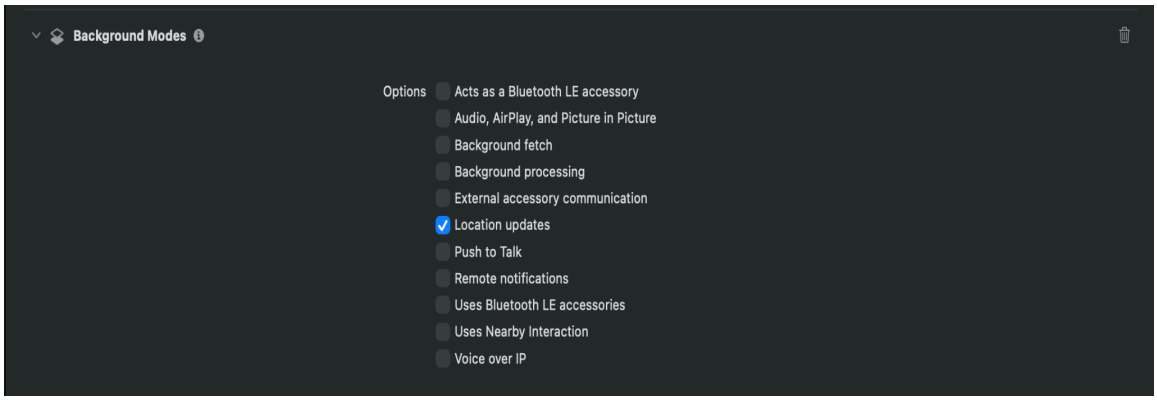
Value: <your app name> needs your location while using the app to capture inspection locations

- If **locationUsage** is **withBackground**

Key: Privacy - Location Always and When In Use Usage Description

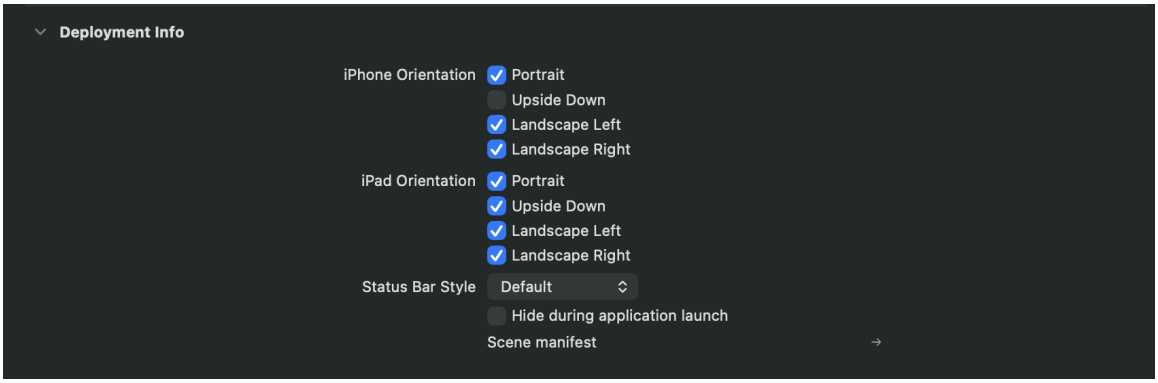
Value: <your app name> needs your location while using the app to capture inspection locations and sync your inspections

Add Background modes support with location updates in the source app if choosing **locationUsage** as **withBackground**



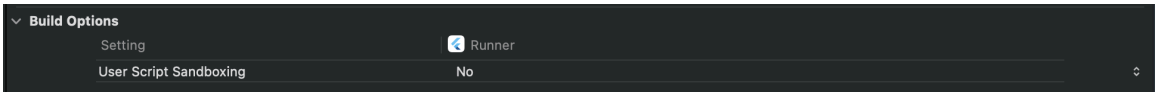
Step 4: Add supported interface orientation

SDK required below supported interface orientation



Step 5: Disable User Script Sandboxing

Make sure app sandboxing is off in build option of target app



Step 6: Add Run Script

Add below Run Script for your app build to upload on App Store

```
set -euo pipefail

VERSION="1.0.0"
MIN_OS="${IPHONEOS_DEPLOYMENT_TARGET}"

patch_plist() {
  plist="$1"
  if [ ! -f "${plist}" ]; then
    return 0
  fi

  if /usr/libexec/PlistBuddy -c "Print :CFBundleShortVersionString" "${plist}" >/dev/null 2>&1; then
    /usr/libexec/PlistBuddy -c "Set :CFBundleShortVersionString ${VERSION}" "${plist}"
  else
    /usr/libexec/PlistBuddy -c "Add :CFBundleShortVersionString string ${VERSION}" "${plist}"
  fi

  if /usr/libexec/PlistBuddy -c "Print :MinimumOSVersion" "${plist}" >/dev/null 2>&1; then
    /usr/libexec/PlistBuddy -c "Set :MinimumOSVersion ${MIN_OS}" "${plist}"
  else
    /usr/libexec/PlistBuddy -c "Add :MinimumOSVersion string ${MIN_OS}" "${plist}"
  fi

  echo "Patched ${plist}"
}

# 1) Patch SourcePackages artifacts so future embeds already have valid keys.
DERIVED_DATA_ROOT="${BUILD_DIR%%/Build/*}"
ARTIFACTS_ROOT="${DERIVED_DATA_ROOT}/SourcePackages/artifacts/tensorflowlitec"
if [ -d "${ARTIFACTS_ROOT}" ]; then
```

```
  find "${ARTIFACTS_ROOT}" -path "*/TensorFlowLiteC*.framework/Info.plist" | while IFS= read -r plist; do
    patch_plist "${plist}"
  done
else
  echo "note: TensorFlowLiteC SPM artifacts not found at ${ARTIFACTS_ROOT}"
fi


# 2) Patch (and re-sign) frameworks already embedded in the app bundle.
FRAMEWORKS_DIR="${TARGET_BUILD_DIR}/${FRAMEWORKS_FOLDER_PATH}"
if [ -d "${FRAMEWORKS_DIR}" ]; then
  for name in TensorFlowLiteC TensorFlowLiteCMetal TensorFlowLiteCCoreML; do
    fw="${FRAMEWORKS_DIR}/${name}.framework"
    patch_plist "${fw}/Info.plist"
    if [ -d "${fw}" ] && [ -n "${EXPANDED_CODE_SIGN_IDENTITY:-}" ] && [
"${EXPANDED_CODE_SIGN_IDENTITY}" != "-" ] && [ "${CODE_SIGNING_ALLOWED:-YES}" != "NO" ];
then
      /usr/bin/codesign --force --sign "${EXPANDED_CODE_SIGN_IDENTITY}"
--preserve-metadata=identifier,entitlements,flags --timestamp=none "${fw}"
    fi
  done
else
  echo "note: Frameworks directory not found at ${FRAMEWORKS_DIR}; skipping embedded patch"
fi
```

4.2 Android

Not Implemented yet

Android ClearQuoteSDK installation for Flutter is not available yet. Installation steps will be documented when Android support is added.

5. Native Bridge Setup

5.1 iOS

Create a Flutter plugin/bridge class that registers MethodChannel and EventChannel handlers and calls ClearQuoteSDK APIs.

Step 1: Create ClearQuotePlugin.swift

Add ios/Runner/ClearQuotePlugin.swift. Core responsibilities:

- Method channel: com.clearquote/sdk
- Event channel: com.clearquote/sdk/events
- Map Dart calls to ClearQuote.shared APIs
- Forward ClearQuoteSDKDelegate.inspectionCompletionStatus to the event sink

Example skeleton:

```
import ClearQuoteSDK
import Flutter
import UIKit

final class ClearQuotePlugin: NSObject {
  static let methodChannelName = "com.clearquote/sdk"
  static let eventChannelName = "com.clearquote/sdk/events"

  private var eventSink: FlutterEventSink?

  func register(with messenger: FlutterBinaryMessenger) {
    let methodChannel = FlutterMethodChannel(
      name: Self.methodChannelName,
      binaryMessenger: messenger
    )
    methodChannel.setMethodCallHandler { [weak self] call, result in
      self?.handle(call, result: result)
    }

    let eventChannel = FlutterEventChannel(
      name: Self.eventChannelName,
      binaryMessenger: messenger
    )
    eventChannel.setStreamHandler(self)
  }

  // Handle: initSDK, startInspection, logout,
  // getDealerCode, isSDKInitialized, manualOfflineSync
}

extension ClearQuotePlugin: FlutterStreamHandler {
  func onListen(withArguments arguments: Any?,
                eventSink events: @escaping FlutterEventSink) -> FlutterError? {
    eventSink = events
    return nil
  }

  func onCancel(withArguments arguments: Any?) -> FlutterError? {
    eventSink = nil
    return nil
  }
}

extension ClearQuotePlugin: ClearQuoteSDKDelegate {
```

```
 func inspectionCompletionStatus(
   identifier: String,
   message: String,
   code: Int,
   isOffline: Bool,
   serverQuoteId: String?,
   serverInspectionId: String?
 ) {
   eventSink?([
     "identifier": identifier,
     "message": message,
     "code": code,
     "isOffline": isOffline,
     "serverQuoteId": serverQuoteId as Any,
     "serverInspectionId": serverInspectionId as Any,
   ])
 }
}
```

Step 2: Register the plugin in AppDelegate

In ios/Runner/AppDelegate.swift, register the plugin with the Flutter engine messenger:

```
import Flutter
import UIKit

@main
@objc class AppDelegate: FlutterAppDelegate, FlutterImplicitEngineDelegate {
 private let clearQuotePlugin = ClearQuotePlugin()

 override func application(
   _ application: UIApplication,
   didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
 ) -> Bool {
   return super.application(application, didFinishLaunchingWithOptions: launchOptions)
 }

 func didInitializeImplicitFlutterEngine(_ engineBridge: FlutterImplicitEngineBridge) {
   GeneratedPluginRegistrant.register(with: engineBridge.pluginRegistry)
   clearQuotePlugin.register(with: engineBridge.applicationRegistrar.messenger())
 }
}
```

5.2 Android

Not Implemented yet

Android native bridge setup for ClearQuoteSDK is not available yet. Kotlin/Java MethodChannel bridging steps will be documented when Android support is added.

6. Flutter (Dart) Usage

Create a Dart wrapper around the method and event channels (for example lib/clear_quote_sdk.dart).

Dart wrapper

```
import 'dart:async';
import 'package:flutter/services.dart';

final class ClearQuoteSDK {
 ClearQuoteSDK._();

 static const _methodChannel = MethodChannel('com.clearquote/sdk');
 static const _eventChannel = EventChannel('com.clearquote/sdk/events');

 static Future<Map<String, dynamic>> initSDK(String key) async {
  final result = await _methodChannel.invokeMethod<Map<Object?, Object?>>(
   'initSDK',
   {'key': key},
  );
  return Map<String, dynamic>.from(result ?? const {});
 }

 static Future<Map<String, dynamic>> startInspection({
  Map<String, String?>? clientAttrs,
  Map<String, dynamic>? inputDetails,
  Map<String, dynamic>? userFlowParams,
 }) async {
  final result = await _methodChannel.invokeMethod<Map<Object?, Object?>>(
   'startInspection',
   {
    'clientAttrs': clientAttrs,
    'inputDetails': inputDetails,
    'userFlowParams': userFlowParams,
   },
  );
  return Map<String, dynamic>.from(result ?? const {});
 }

 static Future<void> logout() {
  return _methodChannel.invokeMethod<void>('logout');
 }

 static Future<String?> getDealerCode() {
  return _methodChannel.invokeMethod<String>('getDealerCode');
 }

 static Future<bool> isSDKInitialized() async {
  final result = await _methodChannel.invokeMethod<bool>('isSDKInitialized');
  return result ?? false;
 }
```

```
  static Future<void> manualOfflineSync() {
    return _methodChannel.invokeMethod<void>('manualOfflineSync');
  }

  static StreamSubscription addInspectionCompletionListener(
    void Function(Map<String, dynamic> status) listener,
  ) {
    return _eventChannel.receiveBroadcastStream().listen((event) {
      if (event is Map) {
        listener(Map<String, dynamic>.from(event));
      }
    });
  }
}
```

Initialize SDK

```
final result = await ClearQuoteSDK.initSDK(sdkKey);
// Success when result['code'] == 200
```

Start Inspection

```
final result = await ClearQuoteSDK.startInspection(
  clientAttrs: {
    'userName': 'Jane',
    'dealer': 'Dealer Name',
    'dealerIdentifier': 'D123',
    'client_unique_id': 'client-1',
    'organisationId': 'org-1',
  },
  inputDetails: {
    'customerDetails': {
      'name': 'Customer',
      'email': 'customer@example.com',
      'dialCode': '+1',
      'phoneNumber': '5550100',
    },
    'vehicleDetails': {
      'regNumber': 'ABC123',
      'make': 'Toyota',
      'model': 'Camry',
      'bodyStyle': 'Sedan',
      'fuelType': 'Petrol',
      'variant': 'XLE',
    },
    'quoteData': {
      'inspectionType': 'standard',
      'fleetImageType': null,
```

```
  },
 },
 userFlowParams: {
  'isOffline': false,
  'skipInputPage': false,
 },
);
```

Listen for inspection completion

```
final sub = ClearQuoteSDK.addInspectionCompletionListener((status) {
 // status: identifier, message, code, isOffline,
 //         serverQuoteId, serverInspectionId
});

// Cancel when done:
// await sub.cancel();
```

Logout / helpers

```
await ClearQuoteSDK.logout();
final dealerCode = await ClearQuoteSDK.getDealerCode();
final initialized = await ClearQuoteSDK.isSDKInitialized();
await ClearQuoteSDK.manualOfflineSync();
```

7. Platform Channel API

Method channel: com.clearquote/sdk

- initSDK — args: { key }. Returns { isInitialized, code, message }
- startInspection — args: { clientAttrs, inputDetails, userFlowParams }. Returns { started, message, code }
- logout — clears SDK session
- getDealerCode — returns String?
- isSDKInitialized — returns bool
- manualOfflineSync — triggers offline inspections sync

More public methods/properties are available in SDK which can be found in below docs:
https://docs.google.com/document/d/1eqHUg3L7mqA4E8vqslzpLoqoC_8qxv7wTUn_JneKQmY/edit?usp=sharing
https://docs.google.com/document/d/1qaoIRasNhM7pLG6hKX2aLnKaZr35R-_8GSMnZpDO9Sw/edit?usp=sharing

Event channel: com.clearquote/sdk/events

Payload fields:

- identifier, message, code, isOffline, serverQuoteId, serverInspectionId

8. Error Codes (Initialization)

Possible messages from ClearQuote SDK while initializing:

- “SDK key is empty” — Recheck if the SDK key is passed correctly
- “Invalid user region” — Recheck if the SDK key is passed correctly
- “SDK Internal Error” — Contact ClearQuote team
- “Success” — ClearQuote SDK initialized successfully

Possible error codes:

- 101 — SDK key is empty
- 100 — Invalid user region
- 500 — SDK Internal Error
- 200 — Success

9. Notes

- Do not initialize the SDK multiple times. Use isSDKInitialized() if required
- Camera and location permissions are mandatory for inspection flows
- Always invoke SDK UI methods on the main thread (the iOS bridge does this)
- Open xcworkspace of your project when working in Xcode so SPM packages resolve correctly

10. Demo App

Below repo is sample flutter app with ClearQuoteSDK integration

https://github.com/clearquotetech/flutter-sample-project

11. Support

If you face issues, raise a GitHub issue with logs and screenshots:

iOS - https://github.com/clearquotetech/cq-ios-sdk/issues

Android - **Not Implemented yet**

For queries:

- sharath@clearquote.io
- bhanusingh@clearquote.io
- akhila@clearquote.io